

Diffusion-Based Time Series Forecasting with Wavelet-Conditioned Score Transformer

This project implements a **score-based generative modeling** framework for multiscale, probabilistic forecasting of time series data. It is a **Transformer-based score network** that generates multiple future trajectories with uncertainty estimates using wavelet decomposed inputs.

Model Architecture

1. Wavelet Decomposition ([data.py](#))

Input time series are decomposed into 5 frequency bands using a level-4 **stationary wavelet transform (SWT)** with the Daubechies-4 ([db4](#)) wavelet. The resulting tensors have shape:

```
$$
[N_{\text{windows}}, T = \text{history\_len} + \text{predict\_len}, 5 \text{ bands}, 1 \text{ feature}]
$$
```

This multiscale representation is used to capture both short- and long-term patterns in the signal.

2. ScoreTransformerNet ([model.py](#))

A custom Transformer-based score network models the conditional score $\nabla_x \log p(x_t \mid x_0)$ in the wavelet domain.

Key Components:

- **Wavelet-scale embedding:** Each of the 5 SWT bands is assigned a learned embedding.
- **Positional encoding:** Separate sinusoidal encodings for historical and future time steps.
- **Time embedding:** A feed-forward network embeds the continuous diffusion timestep t into the Transformer's latent space.
- **Shared Transformer encoder:** Encodes each band independently using shared weights, then fuses them.
- **Decoder stack:** Multi-layer Transformer decoder maps future representations conditioned on history.

Training Objective ([trainer.py](#))

The model is trained using **score matching** under a **variance-preserving SDE (VPSDE)**.

Forward Process (VPSDE)

A noise schedule $\beta(t)$ transforms clean wavelet coefficients x_0 into noisy x_t :

\$\$

$$\mu_t = \alpha(t) \cdot x_0$$

\$\$

\$\$

$$\sigma_t^2 = 1 - \alpha(t)^2$$

\$\$

\$\$

$$x_t = \mu_t + \sigma_t \cdot \epsilon$$

\$\$

where $\alpha(t) = e^{-\frac{1}{2} \int_0^t \beta(s) ds}$ and $\epsilon \sim \mathcal{N}(0, I)$.

Score Prediction

The model predicts the score function:

\$\$

$$s_{\theta}(x_t, t, x_{\text{hist}}) \approx -\nabla_x \log p_t(x_t \mid x_0)$$

\$\$

Total Loss

Two terms are used in training:

1. Score Matching Loss

\$\$

$$\mathcal{L}_{\text{SM}} = \mathbb{E} \left[\left| \sigma_t \cdot s_{\theta}(x_t, t) + \epsilon \right|^2 \right]$$

\$\$

2. Multiscale Reconstruction Loss

A weighted MSE between predicted denoised x_0 and ground truth across wavelet bands:

\$\$

$$\mathcal{L}_{\text{MS}} = \sum_{j=1}^{L+1} \frac{1}{j} \cdot \text{MSE}(x_0^{(j)}, x_{\text{true}}^{(j)})$$

\$\$

In theory this encourages better reconstruction at coarser scales.

Total loss:

\$\$

$$\mathcal{L} = \mathcal{L}_{\text{SM}} + \lambda_{\text{wave}} \cdot \mathcal{L}_{\text{MS}}$$

\$\$



Classifier-Free Guidance (CFG)

Implemented within `ScoreTransformerNet.forward()`, classifier-free guidance interpolates between:

- **Conditional** score (with history)
- **Unconditional** score (with dropped history)

Final score used during sampling:

$$s_{\text{final}} = (1 + w) \cdot s_{\text{cond}} - w \cdot s_{\text{uncond}}$$

Where w is the `classifier_free_guidance_weight` (default: 0.25–3.0).

Time Embedding

Each diffusion step $t \in (0, 1]$ is projected into a learned latent vector using a feedforward MLP:

$$\text{TimeEmbedding}(t) = \text{MLP}(t) \in \mathbb{R}^{\text{model_dim}}$$

This vector is added to all time steps in the Transformer to condition the network on the diffusion timestep.

Sampling (`predictandsave.py`, `sde.py`)

Sampling is performed using **Euler–Maruyama discretization** over the reverse VPSDE:

$$x_{t-1} = x_t + (\text{drift} + \text{score_term}) \cdot \Delta t + \sigma_t \cdot \sqrt{|\Delta t|} \cdot \text{noise}$$

with:

- $\text{drift} = -0.5 \cdot \beta(t) \cdot x_t$
- $\text{score_term} = -0.5 \cdot \sigma(t)^2 \cdot s_{\theta}(x_t, t)$
- $\text{noise} \sim \mathcal{N}(0, I)$

Wavelet coefficients are unnormalized and reconstructed to return space using the approximation band (A4) via:

$$\text{approx_band} = \text{wave}[\text{space} : \text{space} + 1]$$

Evaluation & Forecasting

The model supports probabilistic forecasting with evaluation metrics:

- **CRPS** (Continuous Ranked Probability Score)
- **MAE, RMSE**
- Interval bounds (e.g., 10–90%, IQR)

Returns are reconstructed into prices using log-exp accumulation:

$$\text{price}[t] = \text{price}[0] \cdot \exp\left(\sum_{i=1}^t \text{return}_i\right)$$

File Map

File	Purpose
<code>config.py</code>	Hyperparameters and data paths
<code>data.py</code>	Wavelet loading and <code>WaveletSlidingWindowDataset</code>
<code>model.py</code>	<code>ScoreTransformerNet</code> with positional, scale, and time embeddings
<code>sde.py</code>	VPSDE forward & reverse process
<code>trainer.py</code>	Training loop and loss computation
<code>predictandsave.py</code>	Sampling, plotting, and evaluation
<code>wavelet_unet.py</code>	Optional UNet for wavelet band refinement (unused in final pipeline)

Summary

This diffusion model combines signal processing and modern generative modeling to:

- Learn a time-dependent, multiscale score function in the wavelet domain
- Sample future trajectories using SDE-based generation
- Achieve calibrated, uncertainty-aware forecasting for financial or other structured time series data